# PostgreSQL + Student Lookup

**CSC 425 | macOS | Beginner lab**

**Name: _____ Branch: _____**

You will create a database, add three sample students, and connect it to the Student Lookup already on the Student Dashboard. You will build GET to read students and POST to add a new student. Work through this PDF from beginning to end.

## 1. Get main and create your branch

**Where: VS Code.** You need your existing course repository clone, Git, Node.js 22 or later, and npm. This is a first-time PostgreSQL setup. If PostgreSQL is already installed, ask the instructor to check it before Step 2. Do not delete an existing database.

1. Choose **File > Open Folder** and open your course repository. The Explorer must show both `client` and `server`. This folder is the **repository root**. Keep the repository in a local folder, outside OneDrive, iCloud Drive, and other cloud-sync folders.

2. Choose **Terminal > New Terminal**. Use this terminal for the shell commands in the guide.

3. Type `git status` and press Enter. If it lists unfinished changes, save and commit those on their existing branch first. If they are on `main`, ask for help. Do not discard them.

4. With a clean working tree, run these commands one at a time. Replace `firstname-lastname` with your own name, using lowercase letters and hyphens:

```
git switch main
git pull --ff-only origin main
git switch -c firstname-lastname/student-database
git branch --show-current
```

**Check:** The final command prints your new personal branch. Stay on that branch for this entire lab. Never commit the lab on `main`.

5. Run these commands from the same terminal:

```
npm install
npm run install:all
```

6. Open `.gitignore` in the repository root. Add this line at the bottom and save:

```
server/local-postgres/
```

7. In the Explorer, right-click `server`, choose **New Folder**, and name it `local-postgres`. Create another folder under `server` named `endpointExamples`. If a folder already exists, use it.

**Check:** You have a personal branch and both folders. The database's data will live in `server/local-postgres/data`. Git stores your code; it must not store these live database files or your `.env` password file.

# 2. Get PostgreSQL ready on your Mac

**Already have Postgres.app? Keep it where it is. Start at item 4.** The commands below find it in either `/Applications` or your personal `~/Applications` folder. You do not need to move it between these folders.

**Installing for the first time:**

1. Download the stable app containing **PostgreSQL 18** from Postgres.app downloads. In **Finder > Downloads**, double-click the downloaded `.dmg` file.

2. Open a second Finder window. Press **Cmd+Shift+H** to open your home folder. Open its **Applications** folder. If it is missing, press **Cmd+Shift+N** and name the new folder `Applications`. Drag **Postgres** from the disk-image window into this folder. Wait for copying to finish.

3. Open the copied Postgres app. Confirm **Open** if macOS asks. **Do not click Initialize.** Quit the app with **Cmd+Q**; this lab creates its database storage inside your project instead.

**Find the tools and check the installation:**

4. In the VS Code terminal at the repository root, paste this entire block:

```
PG_TOOLS="Postgres.app/Contents/Versions/18/bin"
export PATH="/Applications/$PG_TOOLS:$HOME/Applications/$PG_TOOLS:$PATH"
psql --version
initdb --version
```

**Check:** Both commands print PostgreSQL **18.x**. The block works with either Applications folder. Keep this terminal open; the setting applies to this terminal. If a command is missing or shows another major version, stop and show the instructor the output.

5. On a **first setup**, create the project database storage:

```
initdb -D server/local-postgres/data -U postgres -W -A scram-sha-256
```

Enter the same new database password at both prompts. No characters appear while you type. Remember this password; it belongs to the database login **postgres**. If `data` already exists, do not run initialization again. Have the instructor check the existing setup before continuing.

6. Start PostgreSQL. Paste both lines together, including the backslash:

```
pg_ctl -D server/local-postgres/data \
  -l server/local-postgres/server.log -o "-p 5432 -k /tmp" start
```

**Check:** You see `server started`. If another server already uses port 5432, ask the instructor to check it; do not delete data or change ports to get past the message.

7. Connect using the same terminal:

```
psql -h localhost -p 5432 -U postgres -d postgres
```

Enter the password from item 5. **Check:** You see `postgres=#`. Continue to Step 3 at that SQL prompt. Do not type the prompt itself.

This lab uses the command-line tools, so Postgres.app's window can stay closed. Postgres.app supports other installation locations with limitations; this lab does not use the affected extension-building features. Installation locations

# 3. Create the database and sample students

**Where: psql, at the `postgres=#` prompt.** Run these two lines separately:

```
CREATE DATABASE student_lookup;
\connect student_lookup postgres localhost 5432
```

Enter your database password if asked. The prompt should begin with `student_lookup`. Now check the actual connection:

```
\conninfo
```

**Check:** It names database **student_lookup**, user **postgres**, host **localhost**, and port **5432**. These are the four values you will put in `.env`. SQL ends with a semicolon; commands beginning with `\` do not. If the database already exists, have the instructor check it; do not delete it.

1. In VS Code, create `server/endpointExamples/setup.sql`. Paste all of the following into the file and save it:

```
CREATE TABLE students (
  student_id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  first_name TEXT NOT NULL,
  last_name TEXT NOT NULL,
  major TEXT NOT NULL
);

INSERT INTO students (first_name, last_name, major)
VALUES
  ('Avery', 'Morgan', 'Computer Science'),
  ('Jordan', 'Bennett', 'Information Systems'),
  ('Taylor', 'Reed', 'Business Administration');

SELECT * FROM students ORDER BY student_id;
```

2. Select all text in the file with **Cmd+A**, then copy with **Cmd+C**. Click inside the psql window and **press Cmd+V**. Press Enter. Run this setup script **once**, not every time you start the app.

**Check:** The result shows three rows: Avery Morgan (ID 1), Jordan Bennett (ID 2), and Taylor Reed (ID 3). If a result viewer shows `(END)`, press `q`. Leave psql by typing `\q` and pressing Enter.

## 4. Save the connection settings

**Where: VS Code Explorer.** Create a file called `.env` directly inside `server`. If it exists, update the entries below. Do not create `.env.txt` or put the file in `client`.

```
PORT=3000
DB_HOST=localhost
DB_PORT=5432
DB_NAME=student_lookup
DB_USER=postgres
DB_PASSWORD="YOUR_DATABASE_PASSWORD"
```

Replace `YOUR_DATABASE_PASSWORD` with the password that just worked in psql; keep the quotes. Save the file. This is your PostgreSQL password, not your computer or GitHub password. Keep `.env` out of Git and screenshots.

# 5. Create the database connection and queries

**Where: VS Code.** Create `server/endpointExamples/studentDb.js`. Paste this entire code block into it, then save:

```
import dotenv from 'dotenv'
import pg from 'pg'

dotenv.config({
  path: new URL('../.env', import.meta.url),
  quiet: true,
})

const pool = new pg.Pool({
  host: process.env.DB_HOST,
  port: Number(process.env.DB_PORT),
  database: process.env.DB_NAME,
  user: process.env.DB_USER,
  password: process.env.DB_PASSWORD,
  connectionTimeoutMillis: 3000,
})

export async function getStudents() {
  const result = await pool.query(`
    SELECT student_id::text AS "studentID",
           first_name AS "firstName", last_name AS "lastName", major
    FROM students
    ORDER BY student_id
  `)
  return result.rows
}

export async function addStudent(firstName, lastName, major) {
  const result = await pool.query(`
    INSERT INTO students (first_name, last_name, major)
    VALUES ($1, $2, $3)
    RETURNING student_id::text AS "studentID",
              first_name AS "firstName", last_name AS "lastName", major
  `, [firstName, lastName, major])
  return result.rows[0]
}
```

**What this file does:** `dotenv` reads your private settings. The pool connects to PostgreSQL. `getStudents()` reads rows. `addStudent()` inserts a row and returns it. PostgreSQL assigns the new ID.

`await` waits for the database. `$1`, `$2`, and `$3` receive the values supplied separately in the array. The field names `studentID`, `firstName`, `lastName`, and `major` match the existing Student Lookup. This file handles data; the next file handles HTTP requests.

**Check:** The filename is `studentDb.js`, with a capital D. It is saved inside `server/endpointExamples`. You do not run this file by itself. Query reference

# 6. Create the Express GET and POST endpoints

**Where: VS Code.** Create `server/endpointExamples/studentServer.js`. Paste this entire block into it and save:

```
import express from 'express'
import { getStudents, addStudent } from './studentDb.js'

export const studentRoutes = express.Router()

studentRoutes.get('/', async (req, res) => {
  const students = await getStudents()
  res.json({ students })
})

studentRoutes.post('/', async (req, res) => {
  const { firstName, lastName, major } = req.body ?? {}
  if (
    typeof firstName !== 'string' || firstName.trim() === '' ||
    typeof lastName !== 'string' || lastName.trim() === '' ||
    typeof major !== 'string' || major.trim() === ''
  ) {
    return res.status(400).json({ message: 'Complete all three fields.' })
  }

  const student = await addStudent(
    firstName.trim(), lastName.trim(), major.trim(),
  )
  res.status(201).json({ message: 'Student added.', student })
})

studentRoutes.use((error, req, res, next) => {
  console.error(error.message)
  res.status(500).json({ message: 'Database request failed.' })
})
```

**GET** calls `getStudents()` and returns `{ "students": [...] }` as JSON. This is the response your frontend already expects.

**POST** reads three fields from `req.body`, checks them, calls `addStudent()`, and returns the saved student with status **201 Created**. Missing or blank fields return **400**.

The final handler sends a JSON error if a database query fails. Express 5 forwards rejected async handlers to it. The SQL remains in `studentDb.js`, so you can see the difference between the database code and the API code.

**Check:** This file has one GET handler and one POST handler. It exports `studentRoutes`. The `/` paths will become `/api/students` when you attach this router in Step 7. Do not add `app.listen()` here. Express router reference

# 7. Attach the routes to the existing server

**Where:** `server/src/app.js`**.** Make these four edits carefully. Do not replace the entire file; the transcript exercise also uses it.

1. At the top, below the existing import lines, add:

```
import { studentRoutes } from '../endpointExamples/studentServer.js'
```

2. Find the old `teachingApi.get('/students', ...)` handler. Delete that entire handler, including its comments. In the current main branch, it looks like this:

```
teachingApi.get('/students', (_request, response) => {
  // Express converts the JavaScript object to JSON for the React frontend.
  // StudentLookup selects the matching full name from this returned array.
  response.json({ students: mockStudents })
})
```

This is the old hard-coded response. Removing it lets requests reach your new database routes. Leave the `mockStudents` array for now; the older `/api/status` route still uses its count.

3. Find the line that starts `teachingApi.use(['/status', '/students', '/transcripts']`. Remove only `'/students',` from its list. The finished statement should be:

```
teachingApi.use(
  ['/status', '/transcripts'],
  cors({ methods: ['GET', 'HEAD', 'OPTIONS'] }),
)
```

4. Lower in the file, find `app.use(express.json())`. Add your router immediately after it. These three lines should now appear together, in this order:

```
app.use(cors({ origin: 'http://localhost:5173' }))
app.use(express.json())
app.use('/api/students', studentRoutes)
```

Save the file. Do not duplicate the existing `cors` or `express.json` lines. `express.json()` reads POST request bodies. The last line connects `/api/students` to the handlers you wrote.

**Check:** Searching `app.js` for `teachingApi.get('/students'` finds nothing. Searching for `app.use('/api/students', studentRoutes)` finds exactly one match. `server/src/index.js` already starts this Express app on port 3000; no extra server is needed.

# 8. Run the app and check GET

**Where: an ordinary VS Code terminal at the repository root.** If you still see a psql prompt, type `\q` and press Enter first.

1. Run:

```
npm run dev
```

Leave this terminal running. It starts Express and the frontend together. **Check:** The API reports `http://localhost:3000`, and Vite reports `http://localhost:5173`. If a port is occupied, stop the earlier copy you started, then retry. Ask for help if you cannot identify it.

2. In your browser, open `http://localhost:3000/api/health`. Confirm it says `"database":"connected"`. Then open `http://localhost:3000/api/students`.

**Check:** You see JSON containing a `students` array with your three database students. Avery's ID is the string `"1"`. If you see five students with IDs 1001-1005, Step 7 still has the old mock handler. If you see `Database request failed.`, confirm PostgreSQL is running and check `server/.env`; restart the API after editing `.env`.

3. Open `http://localhost:5173/#student-lookup`. Enter **Avery Morgan** in **Student full name**, then select **Search**. You should see ID 1 and Computer Science.

## 9. POST a new student, then search again

1. Open a **second VS Code terminal** at the repository root. Type `node` and press Enter. You should see a > prompt. The following block is JavaScript for this Node prompt, not a shell command.

2. Paste the whole block and press Enter. Run it once:

```
var response = await fetch('http://localhost:3000/api/students', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({
    firstName: 'Morgan',
    lastName: 'Ellis',
    major: 'Computer Science',
  }),
})
var result = await response.json()
console.log(response.status, result)
```

**Check:** You see **201**, `Student added.`, and the saved student's ID. A fresh database normally assigns ID 4. Use the returned value if yours differs.

3. In Student Lookup, search for **Morgan Ellis**. Confirm the displayed ID matches the POST response. The React form already calls GET `/api/students`; you do not need to rewrite it. New directory students do not automatically receive a transcript in the separate transcript card.

4. Take a screenshot of the successful lookup. Type `.exit` at the Node prompt when you finish. If you accidentally POST the same name twice, ask the instructor for help; the lookup reports duplicate names.

# 10. Save your work and come back later

**Where: VS Code.** Create `server/endpointExamples/lab-notes-firstname-lastname.md`, using your own name in the filename. Write your branch name, the ID returned by POST, and the POST status code. Add a short explanation of this sequence:

```
Student Lookup -> GET /api/students -> studentServer.js
               -> studentDb.js -> PostgreSQL
               -> rows -> JSON response -> Student Lookup
```

Save your screenshot as `server/endpointExamples/lookup-firstname-lastname.png`. Keep passwords out of it. Then run the following commands in an ordinary terminal at the repository root. Replace `firstname-lastname` with your name:

```
git status
git add .gitignore server/src/app.js
git add server/endpointExamples/setup.sql
git add server/endpointExamples/studentDb.js
git add server/endpointExamples/studentServer.js
git add server/endpointExamples/lab-notes-firstname-lastname.md
git add server/endpointExamples/lookup-firstname-lastname.png
git diff --cached --stat
git commit -m "Connect Student Lookup to PostgreSQL"
git push -u origin firstname-lastname/student-database
```

**Check before committing:** Only your code, SQL, notes, screenshot, and `.gitignore` are staged. `.env` and `server/local-postgres/data` must not appear. Submit your GitHub branch link and screenshot in the course assignment. Do not merge into `main`.

## When you finish today

Press **Ctrl+C** in the terminal running `npm run dev`. In the terminal where you set up the PostgreSQL tools, at the repository root, run:

```
pg_ctl -D server/local-postgres/data -m fast stop
```

## When you return to the project

At the repository root, run this block once to start the stopped database:

```
PG_TOOLS="Postgres.app/Contents/Versions/18/bin"
export PATH="/Applications/$PG_TOOLS:$HOME/Applications/$PG_TOOLS:$PATH"
pg_ctl -D server/local-postgres/data \
  -l server/local-postgres/server.log -o "-p 5432 -k /tmp" start
```

Then run `npm run dev` from the repository root and open `http://localhost:5173/#student-lookup`. Search for Morgan Ellis again. The record remains in PostgreSQL after the API stops.

**Do not repeat initialization, CREATE DATABASE, or the seed INSERT when restarting.** Keep your project in the same folder. Changing Git branches does not reset the database, and pushing code does not upload your local database records.